

Presentation script: RSCI Operations System (SQL prototype)

IT EL 5, Advanced Information Management. About 15 minutes. Lines starting with **Say** are meant to be spoken. Each part says what to show in the **web app**, what to run in **MySQL**, and which rubric words to use.

The rubric at a glance

#	Criterion	Points	Where it is shown	Time
1	Project context	10	Part 1 (slide or this page)	1.5 min
2	Transaction management	15	Part 4: web toggle + <code>sql/03_transactions_demo.sql</code>	2.5 min
3	Database encryption	15	Part 5: Vendors page + <code>sql/06_encryption_demo.sql</code>	1.5 min
4	Authorization and privileges	15	Part 6: sidebar + <code>sql/02_roles.sql</code> , <code>sql/05_revoke_demo.sql</code>	2 min
5	Query optimization	15	Part 7: Expenses page + <code>sql/04_optimization.sql</code>	2 min
6	Integration and technical implementation	15	Parts 2 and 3: architecture + the full pipeline	5 min
7	Presentation	15	The whole script: clear order, case study, live demo	n/a

Total 100.

Setup checklist (before you present)

1. Start **MySQL** in the XAMPP Control Panel (stop it from the panel too, never kill the process).
2. `npm run setup-db` to reset the demo data (about 12 seconds). Tickets 3 and 4 must still be "Approved for PO" for `sql/03`.
3. `npm start`, then open <http://localhost:3000>.
4. Open **three or four private windows** so each stays signed in as its own role: Boss, Engineer, one of PO Officer / Accountant / Inventory, and optionally the Administrator. Demo password for every account is `Password123!`, and the sign-in page has one-click buttons.
5. Open `sql/03` to `sql/06` in phpMyAdmin or MySQL Workbench, and one terminal with the MySQL client.
6. Replace the key in `sql/06_encryption_demo.sql` with `ENC_KEY` from `.env` before running it.

Part 1. Project context (Criterion 1, 10 points)

Say:

Rivera-Sarvida Construction Inc. is a construction company that used QuickBooks plus paper to buy materials. Our system follows one material request from the site engineer all the way to stock in the warehouse. The objectives are: one traceable pipeline, no double approvals or double payments, supplier and payment data that is protected, and every staff role limited to what its job needs.

Users (7 roles):

Role	What they do
Engineer	Files material requests (tickets) and fixes rejected ones
Boss	Approves requests and purchase orders
Operations Manager (OM)	Can also approve purchase orders
PO Officer	Turns an approved request into a purchase order
Accountant	Records the payment and posts it to expenses
Inventory	Receives deliveries and moves stock in and out
Administrator	Read-only oversight of every action (dashboard, activity log, team). Cannot approve, file or move anything, and cannot read the encrypted columns

Business process (this is the rail on the sign-in screen and the Pipeline on the dashboard):

Request -> Boss approval -> Purchase order -> Payment -> Stock

Database requirements (say these as the reasons for the design): relational integrity (foreign keys, unique P.O. numbers), changes that touch several tables must be all-or-nothing, sensitive columns encrypted, privileges per role, and a fast report over a large expense ledger.

Database design: 12 tables in 3NF: `users`, `projects`, `vendors`, `tickets`, `ticket_items`, `purchase_orders`, `po_items`, `payments`, `expenses`, `inventory`, `inventory_log`, `activity_log`. Parent and child pairs (`tickets` and `ticket_items`, `purchase_orders` and `po_items`) show normalization: no repeating item columns.

Part 2. How the system is built (front end, back end, database)

Say:

The system has three layers. The browser is the front end. A Node.js server is the back end. MySQL is the database. Nothing in the browser touches the database directly: every click goes through the server.

FRONT END (browser)			BACK END (Node.js + Express)		DATABASE (MySQL)
public/			server/		rsci_sql
-----			-----		-----
index.html	sign in	fetch	index.js	routes + error mapping	12 tables, InnoDB
app.html	app shell	----->	auth.js	login (bcrypt) + JWT	8 MySQL users (7 roles + login)
js/app.js	all screens	JSON	db.js	one connection pool	stored procedure
css/style.css		<-----	routes/*.js	business rules + SQL	resubmit_ticket
			stock.js	stock in / out helpers	

Front end (what the user sees)

- Plain HTML, CSS and JavaScript. No framework and no build step, so there is nothing to compile.
- `public/index.html` is the sign-in page. `public/app.html` is the shell. `public/js/app.js` draws every screen (dashboard, requests, purchase orders, vendors, inventory, expenses, activity log) and the pop-up forms.
- It only shows what the signed-in role may use: the sidebar menu and the row buttons change per role.
- It never contains SQL. It sends requests like `PATCH /api/tickets/1/decision` and draws the JSON that comes back.

Back end (where the rules live)

- Node.js with Express, plus the `mysql2` driver.
- `auth.js`: checks the email and password (bcrypt), then issues a signed login token (JWT, valid 8 hours).
- `db.js`: keeps **one connection pool per MySQL user**. After login, every query runs as the user for that role, for example `rsci_boss`. This is why the database itself enforces permissions.
- `db.js` also has `withTransaction`: START TRANSACTION, run the steps, COMMIT, or ROLLBACK on any error.
- `routes/*.js`: one file per area. They validate input, run parameterized SQL (safe from SQL injection), and return JSON.

Database (where the data and the final say live)

- MySQL (tested on MariaDB 10.4 from XAMPP). Created by `sql/01_schema.sql`, users and grants by `sql/02_roles.sql`, seed data by `scripts/setup-db.js`.

How one click travels (use this example when asked "how does it work"):

1. Boss clicks **Approve** on a ticket.
2. The browser sends `PATCH /api/tickets/1/decision` with the login token.
3. Express checks the token, sees role = boss, and picks the `rsci_boss` connection pool.
4. The route starts a transaction: `UPDATE tickets ... WHERE status = 'Pending Boss Approval'`, then writes a row to `activity_log`, then COMMIT.
5. MySQL checks that `rsci_boss` is allowed to update that column. The answer goes back as JSON and the screen refreshes.

Part 3. The pipeline live (Criterion 6, 15 points)

Say:

I will follow one request through the whole business process using three roles.

Step	Window	Do this	What the audience sees
1	Engineer	Requests, Submit a ticket . Pick a project, a date, add "Cement 40kg" and "Tie wire" (not in stock), Preview, Confirm & Submit	The availability labels (In stock / Not in inventory yet). Ticket appears as Pending Boss Approval
2	Boss	Dashboard shows it under Waiting on you . Click Reject , type a reason	Status turns Rejected, with the reason
3	Engineer	Edit & resubmit , fix the quantity, resubmit	Same ticket goes back to the Boss, "resubmitted 1x"
4	Boss	Approve	Ticket becomes Approved for PO
5	PO Officer	Create P.O. , enter a unique P.O. number and prices, Save	Ticket becomes PO Created; P.O. is Pending Approval
6	Boss (or OM)	Purchase Orders, Approve	P.O. is Approved
7	Accountant	Confirm purchased , enter a check number	Payment recorded and posted to Expenses in one step
8	Inventory	Receive into stock	Stock goes up, movement log written, ticket Completed
9	Administrator	Dashboard, then Activity Log , filter by user and by action	Every step above appears with who did it and when. There are no action buttons anywhere

Say at the end:

That was eight steps and six working roles, with a different MySQL user behind each. The Administrator saw all of it and could change none of it.

Then open **Activity Log** as the Boss to show the audit trail, and **Inventory** to show the movement log (IN and OUT).

Part 4. Transaction management (Criterion 2, 15 points)

Say:

A transaction is a group of SQL statements that must succeed or fail together. This is ACID: Atomicity, Consistency, Isolation, Durability. For example, paying a purchase order changes three tables: it records the payment, posts an expense, and marks the P.O. as purchased. If the second step fails and the first stays, the books are wrong. So all three run inside one transaction.

Show in the web (Atomicity and ROLLBACK):

1. Tick **Simulate failure before COMMIT** (top right).
2. Create a P.O., confirm a purchase, or stock out. Every statement runs, then the server throws just before COMMIT.
3. The toast says **ROLLED BACK. Nothing was saved.** Refresh: nothing changed.
4. Untick it and repeat: it commits.

Show more errors in the web:

- Reuse a P.O. number: duplicate key error, and the ticket does not change (the ticket update that ran first is rolled back).
- Stock out more than what is on hand: "Not enough stock", no change.
- Double-click **Approve**: the second click gets "not in status ... someone may have just changed it". That is isolation: the `UPDATE ... WHERE status = ...` takes a row lock, so two people cannot both approve.

Show in MySQL: `sql/03_transactions_demo.sql` (run with `mysql --force`, because section B ends in an intentional error):

Section	What it proves	Point at
A	COMMIT: ticket, P.O. and items saved together	<code>START TRANSACTION ... COMMIT</code>
B	ROLLBACK: duplicate P.O. number fails, so the ticket update is undone	<code>SELECT</code> shows ticket 4 still "Approved for PO"
C	Isolation: two sessions, the second waits, then sees 0 rows affected	the comment block
D	Consistency: stock cannot go below zero, guard is in the WHERE clause	1 row affected, then 0 rows affected
E	Stored procedure edit-and-resubmit inside a transaction	<code>CALL resubmit_ticket(...)</code>

ACID in one line each (for the slide):

- **Atomicity**: all steps or none (ROLLBACK demo).
- **Consistency**: rules the database enforces: foreign keys, unique `po_no`, `CHECK (qty > 0)`, `CHECK (qty_on_hand >= 0)`, status guards.
- **Isolation**: row locks from guarded updates (InnoDB).
- **Durability**: once committed, data survives a restart (InnoDB redo log). The database server was restarted several times during development and the committed data was still there.

Part 5. Database encryption (Criterion 3, 15 points)

Say:

We identified three kinds of sensitive data: passwords, supplier tax IDs, and payment check numbers. Passwords are hashed with bcrypt, which is one-way, so nobody can read them back, not even us. Tax IDs and check numbers must be readable by the right people, so they use AES encryption with a key. The key is not in the database. It lives in the server's `.env` file. A copy of the database alone is useless.

Show in the web:

- **Boss** (or PO Officer, Accountant) opens **Vendors**: the TIN column shows the decrypted value.
- **Engineer, OM or Inventory** opens **Vendors**: the TIN column shows *Restricted*.
- Open a paid P.O. as the **Boss**: the pop-up shows the check number. As **Inventory**: *Restricted*.

Show in MySQL: `sql/06_encryption_demo.sql` :

1. Stored value (hex) next to the decrypted value for TINs, check numbers and phone numbers.
2. A wrong key returns `NULL`.
3. `SELECT email, password_hash FROM users` shows bcrypt hashes, not passwords.

Rubric words: `AES_ENCRYPT`, `AES_DECRYPT`, `VARBINARY` columns, key held outside the database, bcrypt for passwords (hash vs encryption).

Part 6. Authorization and privileges (Criterion 4, 15 points)

Say:

Most systems check permissions only in the application. We also check in the database. Every role has its own MySQL user. The server connects as that user, so even if the application had a bug, MySQL would still refuse. This is the principle of least privilege: each user gets only the rights its job needs.

Show in the web:

- Bottom-left of the sidebar shows the role and the MySQL user, for example `rsci_boss`.
- The menu differs per role: the Engineer has no Purchase Orders or Inventory; Inventory has no Expenses.
- Sign in as the **Administrator**: no Submit button, only View on every row, the failure toggle is gone, and a paid P.O. shows its check number as *Restricted*.

Show in MySQL: `sql/02_roles.sql` (read it, do not run it live):

- `CREATE USER` for 8 users: `rsci_auth` (login only) plus one per role.
- **GRANT** examples to point at:
 - `rsci_auth` can read `password_hash`. Nobody else can.

- `rsci_engineer` can read vendors only as `(id, name, contact)`, never `tin_enc` (column-level grant).
- No role has `DELETE`. `activity_log` is insert-only, so history cannot be edited.
- Submitters edit a rejected ticket only through the stored procedure (`GRANT EXECUTE`).
- `rsci_admin` has `SELECT` on every table and nothing else, and the secret columns (`password_hash`, `phone_enc`, `tin_enc`, `check_no_enc`) are left out of its grants. The administrator can oversee everything and still cannot read a secret.

Live denied demo (terminal, signed in as the Engineer's MySQL user; the password is `DB_ROLE_PASSWORD` in `.env`):

```
C:\xampp\mysql\bin\mysql.exe -u rsci_engineer -p rsci_sql
```

```
SELECT id, name, contact FROM vendors;      -- allowed
SELECT tin_enc FROM vendors;                -- denied (column-level)
SELECT * FROM payments;                    -- denied
SELECT password_hash FROM users;           -- denied
DELETE FROM tickets WHERE id = 1;          -- denied
```

REVOKE live: `sql/05_revoke_demo.sql` :

1. Sign in to the web as the **OM** and open **Activity Log** (works).
2. Run `SHOW GRANTS FOR 'rsci_om'@'%'`; , then `REVOKE SELECT ON rsci_sql.activity_log FROM 'rsci_om'@'%'`;
3. Refresh the OM's Activity Log: it now shows "Could not load activity log: SELECT command denied".
4. Run the `GRANT SELECT ...` line to give it back and click **Try again**.

Part 7. Query optimization (Criterion 5, 15 points)

Say:

The Expenses report adds up spending per project for a date range. The ledger has 100,000 rows. Without an index MySQL has to read every row. I will show that with EXPLAIN, add one index, and show the difference.

Show in MySQL: `sql/04_optimization.sql` :

Step	Command	What to point at
1	<code>EXPLAIN SELECT ...</code> before the index	<code>rows</code> about 100,000, <code>Extra = Using where</code> : it examines every row
2	<code>CREATE INDEX idx_expenses_date_proj_amt ON expenses (expense_date, project_id, amount)</code>	A composite covering index: range column first, then the columns the query groups and sums
3	<code>EXPLAIN</code> again	<code>type = range</code> , <code>key = idx_expenses_date_proj_amt</code> , <code>Extra = Using index</code> , <code>rows</code> about 5,000

Step	Command	What to point at
4	The <code>YEAR(expense_date) = 2026 AND MONTH(...)</code> version	The index is ignored: a function on the column is not sargable. Query rewriting (compare the bare column to a range) fixes it

On MySQL 8 use `EXPLAIN ANALYZE` for real timings; on MariaDB use `ANALYZE <query>`.

Show in the web (the same effect, visible):

1. Sign in as the **Boss**, open **Expenses**, click **Run report**. Note the **Query time** under the table (about 75 ms without the index on the demo machine; the very first run can be slower).
2. In MySQL run the `CREATE INDEX` line, click **Run report** again: about 7 ms, roughly ten times faster.
3. To repeat the demo: `DROP INDEX idx_expenses_date_proj_amt ON expenses;`

Rubric words: indexing, `EXPLAIN`, covering index, sargable, query rewriting. (Partitioning and efficient joins are also on the rubric's list; we chose indexing and query rewriting, and say so.)

What the website shows, and what MySQL shows

The web shows the **effects**; the SQL files show the **mechanism**. Use both for each criterion.

What the system identifies	Where it is visible in the web	How to trigger it	Where to show the mechanism
Roles and privileges	Sidebar role and <code>rsci_*</code> user; menu and buttons differ per role	Sign in as different roles	<code>sql/02_roles.sql</code> , denied demo
Revoked privilege	Page error "SELECT command denied ..." with a Try again button	<code>REVOKE</code> in <code>sql/05</code>	<code>SHOW GRANTS</code>
Sensitive data	Vendors TIN and the P.O. check number: decrypted vs <i>Restricted</i>	Compare Boss and Inventory	<code>sql/06_encryption_demo.sql</code> (hex values)
Transaction rollback	Toast "ROLLED BACK. Nothing was saved."	Tick Simulate failure before COMMIT , then save anything	<code>sql/03</code> sections A to B
Integrity rules	Duplicate P.O. number, "Not enough stock", double-approve conflict messages	Try each one	Constraints in <code>sql/01_schema.sql</code>
Isolation	"P.O. is not in status ... someone may have just changed it"	Two windows, approve the same P.O.	<code>sql/03</code> section C
Audit trail	Activity Log page (every approval, payment, stock movement)	Do any action	<code>activity_log</code> table, insert-only grants
Oversight	Administrator dashboard (pipeline, latest actions, team), Activity Log filters	Sign in as Administrator	<code>rsci_admin</code> grants: SELECT only, secret columns excluded

What the system identifies	Where it is visible in the web	How to trigger it	Where to show the mechanism
Query performance	Expenses page shows Query time in ms	Create or drop the index, run the report	<code>sql/04_optimization.sql</code> 1
Business flow	Waiting on you queue, sidebar counters, Pipeline rail	Move one ticket through the steps	The 12-table design

Could be added to the web later (not built):

- A Boss-only panel showing `SHOW GRANTS` for each role. It was removed on request; it would make Criterion 4 visible without leaving the browser.
- An `EXPLAIN` panel next to the Expenses report. It would show the plan change live.
- A small "encrypted" marker on protected fields. Cheap, but adds explanatory wording you asked to keep out of the screens.

Part 8. Closing (Criterion 7, 15 points)

Say:

To summarize: one pipeline, seven roles, twelve tables. Transactions keep every multi-step change all-or-nothing. Sensitive data is encrypted with a key kept outside the database. Each role connects as its own MySQL user with only the rights it needs. And one index turned a full scan of 100,000 rows into a small range scan. Thank you. I am happy to take questions.

Likely questions and answers

Question	Answer
Why encrypt in the database and not only in the app?	The values are stored encrypted, so a database dump or a stolen backup is unreadable without the key. The key is held by the server, not the database.
Why bcrypt for passwords but AES for TINs?	Passwords never need to be read back, so a one-way hash is safer. TINs and check numbers must be shown to authorized people, so they need reversible encryption.
Is AES_ENCRYPT strong enough?	It is a good fit for a school prototype. By default MySQL uses AES in ECB mode; for production you would choose a stronger mode (<code>block_encryption_mode</code> , with an IV) or use transparent data encryption.
What stops a user from using SQL directly?	Their MySQL user has only the grants in <code>02_roles.sql</code> : column-level reads, no <code>DELETE</code> , insert-only audit log. The denied demo shows it.

Question	Answer
Why a stored procedure for editing a rejected ticket?	It lets a submitter change only their own rejected ticket without granting <code>UPDATE</code> or <code>DELETE</code> on the tables. The ownership and status checks are inside the procedure.
What isolation level do you use?	InnoDB's default, REPEATABLE READ. The status-guarded <code>UPDATE</code> takes a row lock, so concurrent approvals serialize.
Why a composite index and in that column order?	The query filters a range on <code>expense_date</code> , so it goes first. <code>project_id</code> and <code>amount</code> are included so the answer comes from the index alone (covering index, <code>Using index</code>).
What would you do if the table were 100 million rows?	Partition by month on <code>expense_date</code> , and keep the covering index per partition.
Why not Firebase like the original site?	The course is about relational and SQL concepts: transactions across tables, <code>GRANT / REVOKE</code> , <code>EXPLAIN</code> and indexes. Firestore is a NoSQL document store with its own rules language and none of those.
What can the administrator do?	Watch everything: the pipeline, every action in the Activity Log with filters, and who did how much. It is read-only at the database level (<code>SELECT</code> only), and the encrypted columns are excluded from its grants, so it cannot read TINs, check numbers or password hashes either.
What did you leave out of the original site?	Billing and variation orders, subcontractors, notifications, photo attachments and Excel export. The core procurement pipeline is complete.

Honest limits (say them before someone asks)

- Developed and tested on MariaDB 10.4 (XAMPP). The SQL avoids MySQL-8-only syntax, but it has not been run on MySQL 8.
- Demo accounts share one password, and the per-role MySQL users share one password. Both are for the demo only.
- The 100,000 expense rows are generated test data.
- Timings depend on the machine; the shape of the result (full scan vs range scan) is what matters.